Optimus-Cost-Agent

Enterprise Architectural Blueprint

Authoritative Gateway-MCP reversal control

Version 2.18

Document control: v2.18

Plan 11.13 — authoritative document publication

Client MCP: P11-FU-9 ACP mcpServers are client supplied; local MCPTrustRegistry validates descriptors before execution.

1. Executive Summary

Conceptual framing adapted from evolutionary software boundaries formulated by Neal Ford, Rebecca Parsons, and Patrick Kua (O'Reilly, 2026). The Optimus-Cost-Agent v2.18 transitions from a basic token-routing utility to a formal Harness Engineering framework designed to govern non-deterministic generation via deterministic software boundaries. Standard AI agents are fundamentally bounded by the Dreyfus model's skill acquisition ceiling; they cannot brute-force their way past the competent or proficient barrier into true architectural consistency through raw scaling alone. Optimus raises the agent's effective architectural competence through deterministic constraints and feedback by wrapping logical model abstraction inside an opinionated local-first Python ACP server governed by automated, multi-tier architectural fitness functions and explicit request-level cost attribution.

2. Harness Engineering & Context Optimization

To protect the token window from context bloat while systematically mitigating hallucination, the agent structure enforces a strict division between Feedforward Controls and Feedback Sensors:

- **Feedforward Controls (Pre-Generation):** Before a prompt payload hits the downstream model, the Context Optimization Node constructs structural boundaries. It strips code down to AST-based functional slices, structures static filesystem metadata layouts at the head of the prompt window to maximise GLM-5.2 input caching, and injects succinct, unambiguous constraints using an Architecture Definition Language (ADL) dialect to minimise context attention allocation.
- **Feedback Sensors (Post-Generation Evaluation):** Once the primary model provider emits a candidate patch, the engine catches the text stream inside a validation loop. It intercepts execution blocks and evaluates them via automated, triggered fitness functions before any source mutations are applied to the active working tree.

3. Automated Architectural Fitness Functions

Optimus treats system governance not as a passive logging layer, but as an active, objective integrity check on core architectural traits. Phase 1 implements highly targeted, local validation primitives organised across five operational planes: Scope (Atomic), Automation (Automated), Invocation (Triggered), Return (Static), and Activation (Proactive). These gates intercept code generations mid-flight to fix structural deviations before changes lock into local Git trees.

4. Data Governance Plane & Local Storage Boundaries

To support enterprise compliance review, data within the local container is sharply bounded. Unparsed source code is strictly prohibited from entering persistent vector indexes. The storage engine uses standard Redis HASH structures to catalogue structural class signatures, summaries, and relative file paths. Numeric performance histories are channelled separately to a Redis TimeSeries cluster (tracking tokens_input, tokens_output, cost_usd) tagged by unique run_id strings and protected by a hard 30-day retention window policy.

5. Request-Level Cost Attribution

5A. Upstream Aggregator Cost Normalization and Single-Key Model

`OPTIMUS_API_KEY` is the only agent-facing credential. It is a local shared secret used to authenticate the agent process to the loopback Optimus Gateway; it is not an upstream vendor key or an Optimus tenant wallet. The Gateway process holds one developer-owned aggregator credential in its own local configuration or OS credential storage. OpenRouter is the default aggregator. Vercel AI Gateway is an allowed second OpenAI-compatible endpoint when its Python integration is modest.

The developer funds the aggregator account directly. The aggregator supplies access to many models, routes across upstream providers, reports normalized usage and USD cost, and debits one developer-owned balance. The local Gateway preserves the one-key, one-budget, one-ledger thesis by isolating that credential from the LLM-driven agent, enforcing policy and budget controls, and recording provider-reported usage.

> There is no Optimus-hosted account, prepaid balance, subscription, tenant, org, project wallet, OAuth/device flow, or public Optimus Gateway.

Normalized cost path

Stage	Credential and responsibility
Local agent	`OPTIMUS_GATEWAY_URL` and `OPTIMUS_API_KEY` only
Loopback Gateway	Shared-secret auth, policy, budget, attribution, aggregator credential isolation
Aggregator	Models, routing, provider-reported billing units and `cost_usd`
Local ledger	Run/session/request/Gateway/provider request attribution

Web search currently shares the OpenRouter credential and balance through a dedicated, deterministic plugin request. Package registry and OSV calls are free public APIs and are not funding paths. OpenTelemetry trace export has no invented per-request charge.

The one-key property for search is explicitly conditional: OpenRouter has deprecated its deterministic plugin, and no deterministic aggregator successor is presently documented. Plugin withdrawal may require a verified-or-fail server-tool design or a standalone search provider with a second key and balance.

6. Deterministic Data-Flow Architecture (Phase 1 MVP)

The updated system design integrates closed-loop Harness Engineering directly into the orchestration sequence:

1. User Request Initiated (IntelliJ IDEA Client).
2. Ingress over ACP Protocol over StdIn JSON-RPC to the Optimus ACP Server Daemon (Asyncio Pipeline).
3. Read Topology & Structural Memory Maps from Local Redis State Store (HASH Schema Core Pull), which returns active code signatures and cached map assets.
4. Processing inside the Context Optimization Node [FEEDFORWARD CONTROLS] executing AST Slicing, Polyglot Fallbacks, ADL Constraint Prompt Injection, and Prompt Anchoring for Cache Tuning.
5. Emits minimal context prompt payload to the LangGraph Stateful Router (Claude Haiku 4.5 Triage Gate) for escalation route evaluation.
6. **Delivery to the loopback Optimus Gateway -> approved upstream aggregator (OpenRouter by default; Vercel AI Gateway if retained), returning the model output and provider-reported usage and USD cost. Model aliases remain policy inputs, but the agent never selects or contacts a direct provider adapter.**
7. Validation inside the Automated Fitness Function Engine [FEEDBACK SENSORS] conducting atomic structural checks via PyTestArch and static code metrics gate assertions (MI / Complexity Check). On FAIL: loops back to Step [6]; on PASS: approves code patch execution path.
8. Storage inside the Local Storage Engine Persistence Layer via Async TS.ADD Numeric Telemetry Tracking (RedisTimeSeries) and Async HSET Workspace Metrics Serialisation (Redis HASH Architecture Indexes).
9. Real-Time FinOps Console Panel display for IDE Cost Dashboard Live Update Screen.

7. Agent Operating Modes & Trust Framework

To guarantee user trust and operational predictability, the control plane implements strict execution state isolation:

- **Plan/Chat Mode:** Advisory-only mode. The agent may inspect context, discuss requirements, propose plans, and produce diffs or snippets for review, but it must not mutate the repository, filesystem, external services, or user/project state. Internal cost, audit, and performance telemetry is append-only and explicitly allowed.
- **Agent Mode:** Execution mode. The agent is write-authorised and empowered to actively modify code, run tools, create files, update tests, and apply patches within approved workspace boundaries. Modifications apply to the working tree only after clearing all composite fitness engine gates.
- **Code Generation Scope Classification:** Delineates advisory inline fragments from implementation deliverables based on clear architectural criteria:
    - `INLINE_SNIPPET`: Explanatory text fragments under 15 lines that touch zero core packages and create/delete no files.
    - `PATCH_PROPOSAL`: A bounded diff or file mutation proposal for review.
    - `IMPLEMENTATION_DELIVERABLE`: A multi-file change that requires Agent Mode and all release gates.

– **PATCH_PROPOSAL:** Token streams over 15 lines that edit existing code assets exclusively without file structural modifications.

– **FILE_MUTATION:** Single-file generation operations, re-writes, or file deletions that require shell checking or tool alignment.

– **MULTI_FILE_CHANGESET:** High-risk operations touching shared/core architecture modules, crossing distinct directory roots, or altering multi-package configuration environments.

8. Tool Governance & Evidence Acquisition

Optimus treats tool use as policy-driven first, model-requested second: the model may ask for evidence, but the harness alone decides whether a tool is allowed, necessary, and cost-justified. This keeps tool invocation aligned with the same cost-control discipline applied to model routing.

Tool Stack (Phase 1)

Tool Class	Examples	Invocation Style
Local repo discovery	rg, file reader, AST parser, dependency graph, git diff	Deterministic, cheap, first-line
Validation gates	Tests, lint, type check, architecture fitness checks, security scan subset	Deterministic after patch generation
Patch workspace	Shadow apply, diff parser, rollback, working-tree apply	Agent mode only
Cost telemetry	RedisTimeSeries, run metadata hash, pricing snapshot	Always-on internal telemetry
Web evidence	Gateway search + extract (Tavily-backed)	Policy-triggered, read-only
Package/version metadata	PyPI, npm, Maven Central, GitHub releases, OSV.dev	Only when dependency/version/security task requires it
External execution	Shell, build, test, package install	Agent mode only, gated

Deterministic Invocation Policy

A ToolInvocationPolicy matrix maps task signals to permitted tool classes, so "search the web" is never a casual model impulse. Representative triggers:

- User explicitly asks for current/latest/external facts → web search.
- Task mentions library/API/framework behaviour not present in the repo → official-doc web search.
- Task touches dependencies → package registry lookup.
- Task touches security/CVEs → OSV or advisory lookup.
- Task asks for a local code change → local repo search first, no web by default.
- Model makes an architectural claim without inspected evidence → file inspection or Assumption Ledger entry.
- Patch generated → shadow apply + fitness gates.

- Fitness gate fails → reflection or targeted local inspection.
- Plan/Chat mode active → read/search tools only, no mutation tools.

Each web-evidence request carries an explicit reason code (e.g. API_DOCS_OUTDATED, CURRENT_FACT, SECURITY_ADVISORY, USER_REQUESTED, PACKAGE_VERSION) plus an allowed-domain list, result cap, and search-depth setting, so external calls remain auditable and cost-bounded rather than ad hoc.

Web evidence acquisition is wrapped behind Optimus-owned typed tools rather than exposing a third-party search API directly to the model. This lets the harness enforce allowed domains, timeouts, retries, result caps, and cost telemetry uniformly. Key rule: local evidence first, external evidence only when policy-triggered, mutation only after mode and fitness gates pass.

9. Adaptive Agent Execution Strategy & Rigor Policy

Optimus applies adaptive execution strategies to balance cost, rigor, and reliability. The agent selects Direct Execution, Plan-and-Execute, ReAct, or Reflection based on task ambiguity, mutation risk, and validation outcomes. Reflection and deep planning are not default behaviours; they are exception-triggered circuit breakers invoked only when risk signals, failed validation, or insufficient evidence require additional rigor.

Low-Cost Hallucination Controls

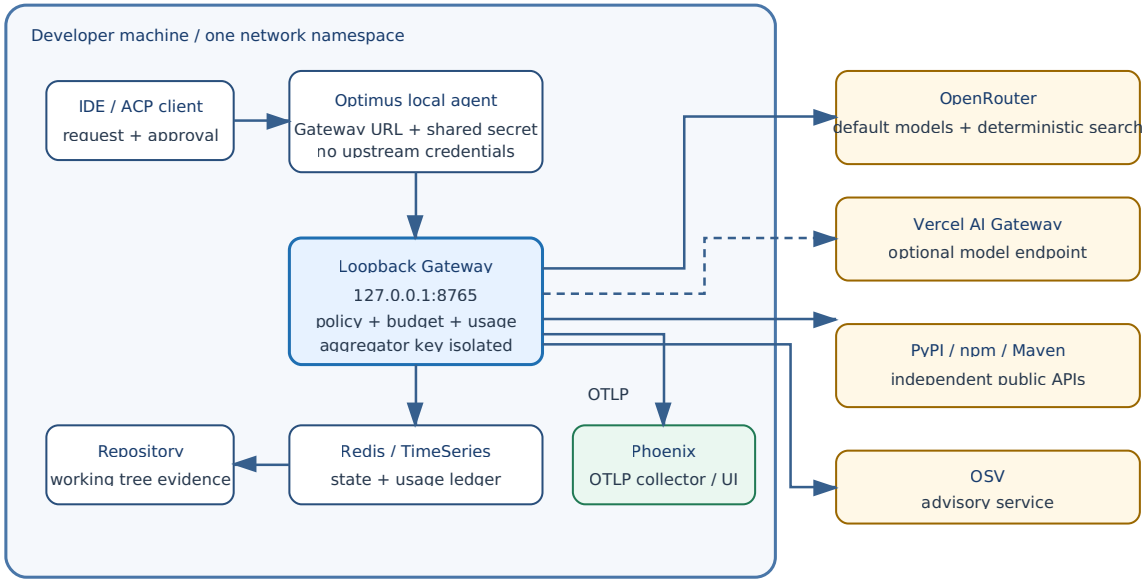
- **Evidence-Bound Decisions:** No architectural claim may be promoted to an implementation decision unless backed by inspected context or recorded as an explicit entry in a machine-readable Assumption Ledger.
- **Rigor Budget Matrix:** Bounded request tiers (LOW, MEDIUM, HIGH) prevent the agent from spending tokens on ceremonial planning loops for simple edits:
  - **LOW:** Cap at 5 tool calls, 0 reflection passes, max 100k context tokens, preferred HAIKU model tier.
  - **MEDIUM:** Cap at 10 tool calls, 1 reflection pass, max 150k context tokens, preferred PRO model tier.
  - **HIGH:** Cap at 15 tool calls, 3 reflection passes, max 250k context tokens, preferred PRO model tier.
- **Trigger-Based Reflection:** Reflection initialises only when validation loops trip a failure signature or code touches shared/core architecture modules, preventing quiet token-doubling on every nominal request.
- **Assumption Ledger:** Bypasses token-heavy text summaries by generating an explicit JSON ledger identifying unverified assumptions, confidence thresholds, and specific validation files needed to clear uncertainty.
- **The Escalation Ladder:** Bounds agent tool invocation paths to escalate strictly through a cost-minimising chain: Provided Context → File Inspection → Plan → Execute → Validate → Targeted Reflection → User Escalation.

10. Architectural Control Flow

This section provides visual reference diagrams for the three primary control flow concerns in Optimus: where the system boundaries sit, how the governance loop operates end-to-end, and

10.A System Context Diagram

The IDE, local agent, loopback Gateway, Redis, repository, and optional Phoenix collector are inside one developer environment. The agent and Gateway are separate processes with separate environments. The agent authenticates with a local shared secret and cannot read the Gateway's aggregator credential.



System context: local Gateway and its controlled external edges

The Gateway alone may contact the approved upstream aggregator, package registries, OSV, and the configured OTLP collector. An agent in WSL2 cannot reach a Windows-host Gateway through loopback; Phase 1 requires both processes in the same network namespace.

Figure 10.A - Local-first system context. No MCP endpoint is shown or implied.

B. Governance Loop Diagram

Every user request traverses the full governance loop. The loop enforces that no model output reaches the working tree without passing through both the feedforward constraint layer and the feedback fitness gate layer.

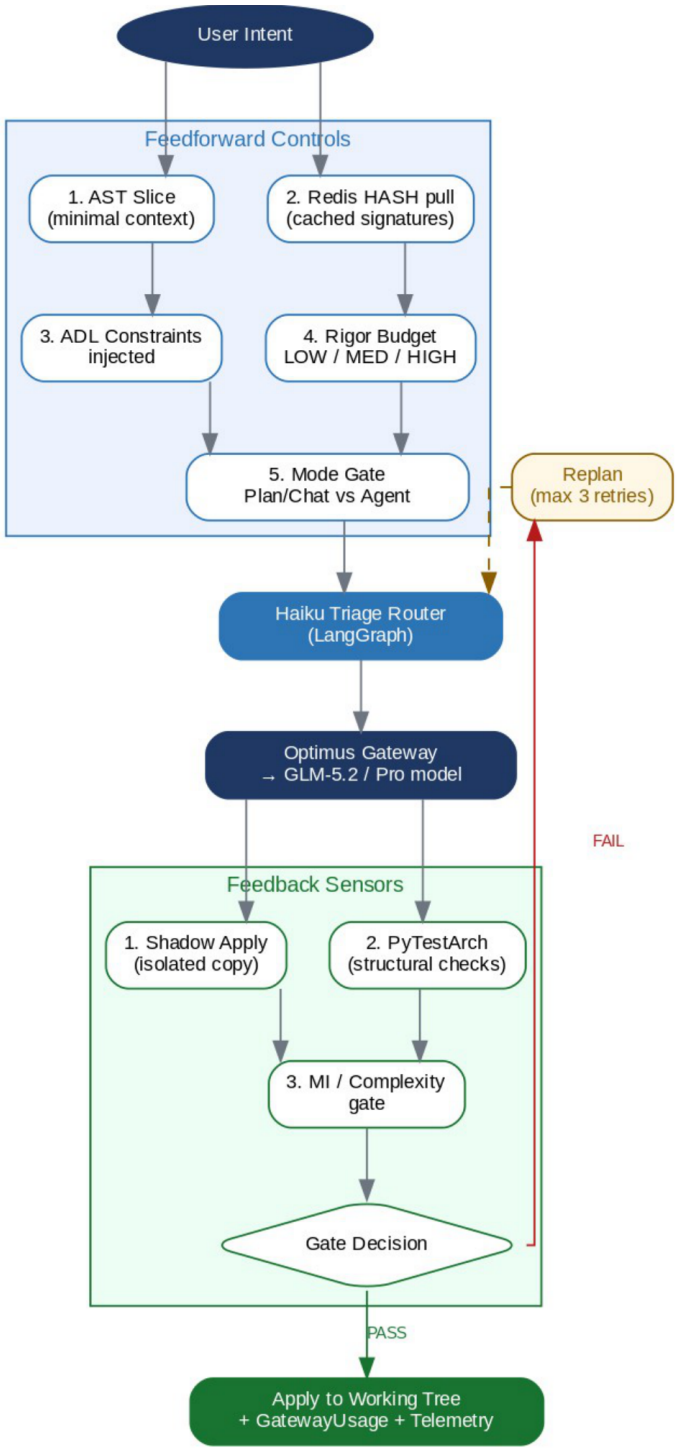


Figure 10.B · Governance Loop — feedforward controls, triage routing, feedback sensors, mutation gate.

10.C Phase Evolution Diagram

Optimus is designed as a three-phase architecture. Phase 1 is mandatory before any later phase.

PHASE 1 -> Python Local Agent + Optimus Gateway Current - Mandatory before Phase 2

Release Gate: the agent runs with only `OPTIMUS_GATEWAY_URL` and `OPTIMUS_API_KEY`; no upstream key is resolvable in the agent process. The separate loopback Gateway receives only its approved aggregator key and optional infrastructure configuration.

PHASE 2 -> Rust Core Rewrite Trigger: Python performance ceiling or team scaling

PHASE 3 -> Agentic Mesh Topology Trigger: Multi-repo / multi-team deployment

See: Optimus-Architecture-Roadmap.pdf

10.D Where Cost Control Happens

Cost control is not a single checkpoint - it is enforced at four distinct points in the request lifecycle, each with a different mechanism.

Control Point	Mechanism	What It Prevents
1. Pre-prompt (Feedforward)	AST slicing + cache anchoring + ADL constraints	Unnecessary tokens sent to expensive models.
2. Router (Triage Gate)	Haiku classifies task -> routes to cheapest sufficient model	Over-routing simple tasks to Pro/Opus tier.
3. Rigor Budget (Runtime)	LOW/MEDIUM/HIGH tier caps tool calls + reflection passes	Token-doubling on ceremonial planning loops.
4. Post-call (Attribution)	Gateway usage object -> EvidenceLedger -> RedisTimeSeries	Silent cost accumulation; enables per-run audit.

10.E Where Hallucination Control Happens

Hallucination is controlled through a defence-in-depth stack. No single gate is sufficient; all five layers must be active simultaneously.

Layer	Mechanism	What It Catches
1. Context Constraint	ADL rules injected pre-prompt; AST slices only relevant code	Irrelevant context that confuses the model.
2. Evidence Gate	Assumption Ledger: every architectural claim must be inspected or logged	Unverified claims promoted to implementation decisions.
3. Tool Policy	ToolInvocationPolicy blocks casual web calls; reason codes required	Model hallucinating external facts without evidence.
4. Fitness Functions	PyTestArch and static metrics gates	Structurally invalid or low-quality changes.
5. Reflection Circuit	Bounded reflection with prior-failure summaries	Repeated failure without targeted replanning.

11. Optimus Gateway - Phase 1 Local Process Boundary

The Optimus Gateway is a deterministic loopback process run by the developer alongside the local agent. It is not a hosted Optimus service, tenant control plane, subscription product, or central wallet.

Gateway responsibilities

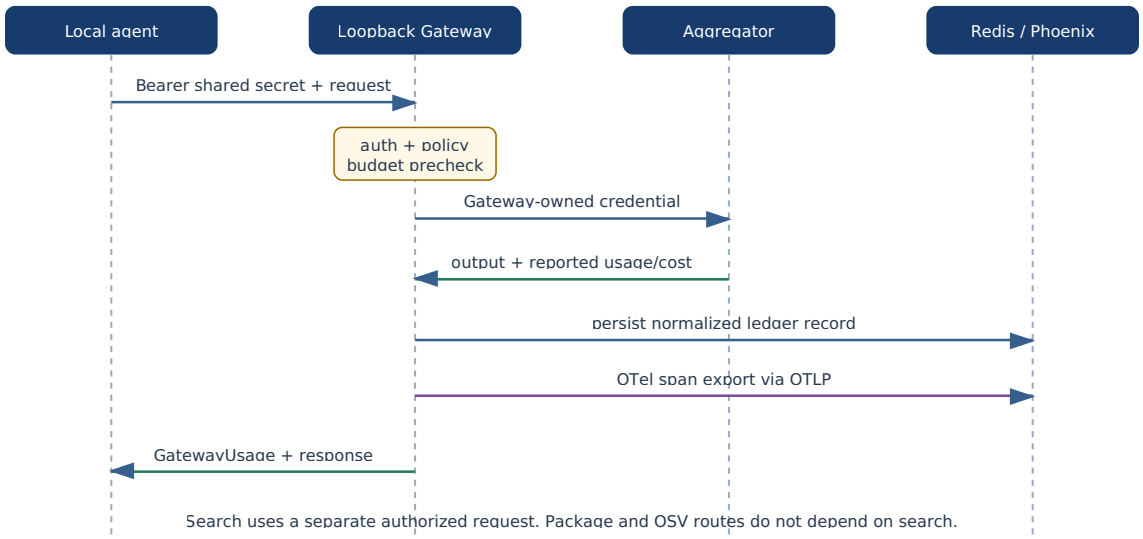
- Bind to strict loopback and authenticate the agent with a local shared secret.
- Isolate the developer's aggregator credential in Gateway-owned local configuration or OS credential storage.
- Route approved model aliases through the surviving OpenAI-compatible transport.
- Enforce model/tool policy, domain rules, call caps, provenance, and the current run's USD budget.
- Return provider-reported usage and cost in a validated GatewayUsage envelope.
- Broker deterministic search, bounded extract, package lookup, and OSV advisory independently.
- Accept structured trace ingress, map it to OpenTelemetry, and export OTLP.
- Persist usage attribution across run, session, request, Gateway request, and provider request IDs.

Process-scoped configuration

Agent process environment	Gateway process local configuration
`OPTIMUS_GATEWAY_URL=http://127.0.0.1:8765`	`OPTIMUS_LOCAL_GATEWAY_PROVIDER=openrouter`
`OPTIMUS_API_KEY=<local shared secret>`	`OPTIMUS_LOCAL_GATEWAY_PROVIDER_API_KEY=<developer key>`
No provider, search, or OTel credentials	`OPTIMUS_LOCAL_GATEWAY_SHARED_SECRET=<same secret>`
Loopback URL only	allowed domains, Redis URL, optional OTLP endpoint
No hosted-origin override	no tenant profile, Vault, or non-loopback mode

Run the agent and Gateway in the same network namespace. A Windows-host Gateway is not loopback from an agent running inside WSL2.

11.1 Gateway request / response sequence



Request sequence from agent to loopback Gateway, aggregator, ledger, and Phoenix

The agent sends one authenticated request to the loopback Gateway. The Gateway applies policy and budget controls, calls the configured aggregator with its Gateway-owned credential, parses provider-reported usage and cost, and returns the normalized GatewayUsage envelope. The aggregator credential never enters the agent process or response.

Search follows a separate authorized request and annotation path. Package and OSV routes do not depend on search availability. Trace delivery is an independent Gateway-to-OTLP operation and does not create a fabricated per-request observability charge.

11A. OpenTelemetry Trace Observability

Phase 1 uses OpenTelemetry spans and OTLP as the vendor-neutral trace contract across planning, Gateway calls, tool invocation, validation, retries, and final response generation. The local agent sends authenticated structured trace ingress to the Gateway; the Gateway validates and redacts fields, maps them to OTel spans/events, and exports OTLP. Arize Phoenix is the documented local default. Any OTLP-compatible backend may replace it without changing Optimus instrumentation; Langfuse may be considered for team-scale deployments.

Trace export records operational telemetry, not an invented billable provider request. No allocated or amortized observability `cost_usd` is added to the usage ledger. Infrastructure cost is an operator concern outside per-request accounting.

Required attributes retain run, session, request, Gateway/provider request, execution mode, generation scope, model/provider, cache, `cost_usd`, billing units, policy/tool, validation, retry, and failure attribution. Secrets are redacted before export.

12. Testing and quality gates

Concern	Tooling	Role in gates
Code coverage	`coverage.py`, `pytest-cov`	Release gate: at least 80%, safety-critical modules do not regress
Agent quality eval	DeepEval, Ragas	Task quality, correctness, groundedness
Adversarial/red-team	PyRIT	Prompt injection, tool-control, and trust-boundary validation
Trace observability	OpenTelemetry + OTLP; Phoenix default	Debugging and regression analysis, not a coverage metric

OTel/OTLP trace validation is tracked separately from production-code coverage. LangSmith is not part of the architecture or dependency set.

13. Agent Execution Safety & Guardrails

Agent execution safety is enforced through a layered guardrail model that wraps every tool call between the moment the agent decides to act and the moment the action takes effect: **mode constraints** (§7), a **permission policy** (allow/deny lists with a human-approval path), a **pre-tool guard** that deterministically validates shell, file, MCP, and network calls, **sandboxed execution**, **post-tool audit** against the Gateway cost and trace ledgers (§5A, §11, §11A), and **commit/CI gates** (§12). The same plane governs two execution patterns — **bounded goal-driven agent loops** and **curated workflow skills** — so that long-running or procedure-driven work inherits the identical constraints and the same Gateway budget policy.

The premise is not that the agent is malicious but that it is a reward-hacky problem solver: under pressure to reach "done" it will reach for `chmod 777`, `curl … | sh`, force-push to `main`, or commenting out a failing assertion. Guardrails remove those shortcuts from its reach. Most of the model is deterministic and zero-token; a cheap, budgeted classifier routed through the Optimus Gateway is consulted only for borderline cases, never on the hot path of an already-allowed or already-denied call.

The detailed policy model, component contracts, and control-flow diagram are maintained in the companion specification **Agent Execution Guardrails & Workflow Strategy v1.0** (control-plane diagram in its §1; LLD component contracts in its §10; test categories in its §11). The corresponding implementation contracts are specified in **LLD §12** and the test categories in **Test Strategy §14**. This section is a cross-cutting reference only.